

Trabalho Prático Semestral - Peso 6

Sistema de Gerenciamento de Eleições

API REST com Django REST Framework

INSTRUÇÕES — Esta atividade é INDIVIDUAL e tem duração de 3 horas corridas. Cada etapa de avaliação deve ser entregue como um commit separado (ver Seção 7).

Link do GitHub Classroom:

<https://classroom.github.com/a/KEr3YAoF>

1. Contexto do Projeto

Você foi contratado para projetar e implementar, do zero, uma API RESTful completa para um Sistema de Gerenciamento de Eleições. O sistema deve permitir a criação de eleições de qualquer tipo (estudantil, condomínio, sindical, associação, conselho institucional etc.), o cadastro de candidatos e o gerenciamento dos eleitores aptos a votar.

O ponto mais delicado do projeto é o SIGILO DO VOTO: o sistema precisa garantir, simultaneamente, que: (i) cada eleitor vote apenas uma vez por eleição e (ii) seja impossível associar um voto registrado ao eleitor que o emitiu. Além disso, cada votante deve receber um comprovante de votação contendo um QR Code com data, horário e candidato escolhido — acessível apenas pelo próprio eleitor, no momento do voto.

ATENÇÃO — Não pode existir qualquer FK (ForeignKey), índice composto, log ou qualquer outro recurso que permita reconstruir a relação entre o eleitor e seu voto. Violar essa regra implica em perda automática de pontos na Questão 02 (modelagem) — independentemente do resto do projeto.

1.1 Regras de Negócio do Sistema

- Uma eleição possui período de votação (data/hora de início e fim) e um status (rascunho, aberta, encerrada, apurada).
- Uma eleição possui 2 ou mais candidatos. Não é permitido abrir eleição com menos de 2 candidatos.
- Cada eleitor é vinculado a uma ou mais eleições via lista de aptidão (apenas eleitores aptos podem votar em determinada eleição).
- Um eleitor só pode votar UMA vez por eleição.
- O voto é ANÔNIMO: a tabela de votos não pode conter qualquer referência ao eleitor.
- A opção "voto em branco" deve ser permitida em todas as eleições.
- O comprovante de voto é entregue UMA ÚNICA VEZ na resposta do endpoint de votação. O sistema armazena apenas o hash do token de comprovante.

- Após o encerramento da eleição, ninguém — nem mesmo o administrador — pode emitir novos votos ou alterar votos existentes.
- Relatórios públicos exibem percentual de votos por candidato, total de votantes e abstenções; nunca exibem qualquer dado individualizado.

2. Estratégia de Anonimato do Voto

Para conciliar as exigências de 'um voto por eleitor' e 'voto irrastrável', você deve adotar a estratégia de DESACOPLAMENTO entre o ato de comprovação (eleitor participou) e o ato de voto propriamente dito (qual candidato recebeu o voto).

Modelo conceitual obrigatório

Entidade	Função
RegistroVotacao	Sabe que o eleitor X já votou na eleição Y. NÃO sabe em quem ele votou. Atende às regras 'um voto por eleitor' e 'lista de quem compareceu'.
Voto	Sabe apenas: a qual eleição pertence, em qual candidato (ou se foi em branco) e quando foi registrado. NÃO sabe quem o emitiu — não possui qualquer FK para Eleitor.

O endpoint de votação deve executar duas operações:

1. Criar um RegistroVotacao (eleitor + eleição), respeitando a constraint UNIQUE TOGETHER.
2. Criar um Voto (apenas eleição + candidato + data/hora + hash do comprovante).

Estratégia do Comprovante e do QR Code

- No ato do voto, gere um token aleatório seguro: token = secrets.token_urlsafe(32).
- Salve APENAS o token_hash no campo Voto.comprovante_hash. O token em texto plano não fica no banco.
- Retorne, na resposta do endpoint /votar/, o token original junto com a URL do QR Code.

3. Especificação do Modelo de Dados

Implemente as entidades a seguir. Preste atenção redobrada aos relacionamentos, aos índices de unicidade (unique e unique_together) e às restrições de on_delete.

3.1 Entidade: Eleitor

Campo	Tipo Django / Observação
nome	CharField(max_length=150)
email	EmailField(unique=True)

Campo	Tipo Django / Observação
cpf	CharField(max_length=14, unique=True) — Formato 000.000.000-00
data_nascimento	DateField()
ativo	BooleanField(default=True)
data_cadastro	DateTimeField(auto_now_add=True)

3.2 Entidade: Eleicao

Campo	Tipo Django / Observação
titulo	CharField(max_length=200)
descricao	TextField(blank=True)
tipo	CharField com choices — estudantil, sindical, associacao, condominio, conselho, outra
data_inicio	DateTimeField()
data_fim	DateTimeField()
status	CharField com choices — rascunho, aberta, encerrada, apurada (default: rascunho)
permite_branco	BooleanField(default=True)
criada_por	ForeignKey(Eleitor, on_delete=PROTECT, related_name='eleicoes_criadas') — administrador responsável

OBSERVAÇÃO — Em `models.clean()`, valide que `data_fim > data_inicio` e que o status só pode ser alterado seguindo o fluxo: rascunho → aberta → encerrada → apurada (sem voltar).

3.3 Entidade: Candidato

Campo	Tipo Django / Observação
eleicao	ForeignKey(Eleicao, on_delete=CASCADE, related_name='candidatos')
numero	PositiveIntegerField() — Número de exibição do candidato
nome	CharField(max_length=150)
nome_urna	CharField(max_length=50)
partido_ou_chapa	CharField(max_length=100, blank=True)
proposta	TextField(blank=True)

Campo	Tipo Django / Observação
foto_url	URLField(blank=True)

OBSERVAÇÃO — Defina Meta: `unique_together = [('eleicao', 'numero')]` — números são únicos POR eleição.

3.4 Entidade: AptidaoEleitor

Relaciona eleitores aptos a votar em uma determinada eleição.

Campo	Tipo Django / Observação
eleitor	ForeignKey(Eleitor, on_delete=PROTECT, related_name='aptidoes')
eleicao	ForeignKey(Eleicao, on_delete=CASCADE, related_name='aptos')
data_inclusao	DateTimeField(auto_now_add=True)

OBSERVAÇÃO — Defina Meta: `unique_together = [('eleitor', 'eleicao')]` — um eleitor só pode ser cadastrado uma vez como apto em cada eleição.

3.5 Entidade: RegistroVotacao

Esta tabela é o 'caderno de presença' da eleição. Sabe quem compareceu, mas não em quem votou. Permite responder consultas do tipo 'o eleitor X já votou na eleição Y?' sem expor o voto.

Campo	Tipo Django / Observação
eleitor	ForeignKey(Eleitor, on_delete=PROTECT, related_name='registros_votacao')
eleicao	ForeignKey(Eleicao, on_delete=PROTECT, related_name='registros_votacao')
data_hora	DateTimeField(auto_now_add=True)

OBSERVAÇÃO — Defina Meta: `unique_together = [('eleitor', 'eleicao')]` — garantia em nível de banco de que ninguém vota duas vezes na mesma eleição.

3.6 Entidade: Voto

Esta é a entidade central do sistema. Esta tabela NÃO pode possuir qualquer FK para Eleitor.

Campo	Tipo Django / Observação
eleicao	ForeignKey(Eleicao, on_delete=PROTECT, related_name='votos')

Campo	Tipo Django / Observação
candidato	ForeignKey(Candidato, on_delete=PROTECT, related_name='votos', null=True, blank=True) — nulo quando o voto é em branco
em_branco	BooleanField(default=False)
data_hora	DateTimeField(auto_now_add=True)
comprovante_hash	CharField(max_length=64, unique=True) — SHA-256 do token entregue ao eleitor

OBSERVAÇÃO — Em `models.clean()`, valide que (`em_branco=True` E `candidato=None`) OU (`em_branco=False` E `candidato is not None`). Os dois casos mutuamente exclusivos.

4. Questões da Atividade

OBSERVAÇÃO — Cada questão a seguir corresponde a UM commit obrigatório no repositório, com a mensagem padronizada indicada em cada questão. **A ordem dos commits será conferida na avaliação.**

Questão 01 — Configuração do Projeto e Modelagem

Commit esperado: setup do projeto e modelos iniciais

Configure o projeto Django completo para a API de eleições. Siga os passos abaixo:

3. Crie o projeto Django chamado **eleicoes_api** e o app interno chamado **urna**.
4. Configure o banco de dados PostgreSQL no settings.py, criando o banco **eleicoes_db**.
5. Registre os apps rest_framework, drf_yasg e urna no INSTALLED_APPS.
6. Crie TODOS os modelos especificados na Seção 3 (Eleitor, Eleicao, Candidato, AptidaoEleitor, RegistroVotacao, Voto) no arquivo urna/models.py.
7. Implemente os métodos clean() de Eleicao e Voto com as validações descritas na Seção 3.
8. Execute makemigrations e migrate.
9. Cadastre via Django Admin um cenário de testes mínimo: 1 administrador (*superuser*), 5 eleitores comuns, 1 eleição com 3 candidatos e 5 eleitores aptos.

Questão 02 — Serializers

Commit esperado: serializers da API

Crie os Serializers no arquivo urna/serializers.py conforme as especificações abaixo:

10. EleitorSerializer — todos os campos. O campo cpf deve aceitar entrada apenas no formato 000.000.000-00 (use regex no validate_cpf).
11. EleicaoSerializer — inclua os campos calculados: status_display (get_status_display), total_candidatos (SerializerMethodField que conta candidatos da eleição), total_aptos (count em aptos).
12. CandidatoSerializer — inclua eleicao_titulo (source='eleicao.titulo', read_only=True). Valide que o numero não seja zero (zero é reservado para voto em branco na exibição de relatórios).
13. AptidaoEleitorSerializer — inclua eleitor_nome (source='eleitor.nome') e eleicao_titulo (source='eleicao.titulo'); ambos read_only.
14. RegistroVotacaoSerializer — apenas leitura (não será criado via POST direto), inclua eleitor_nome e eleicao_titulo.
15. VotoSerializer — read-only para fins de relatório. Inclua candidato_nome_urna (source='candidato.nome_urna', read_only=True, allow_null=True) e em_branco_display (SerializerMethodField que retorna 'BRANCO' se em_branco=True, senão None). O campo comprovante_hash NUNCA deve ser exposto nas respostas (defina como write_only ou exclua de fields).
16. VotacaoInputSerializer — serializer dedicado APENAS para o endpoint /votar/. Campos de entrada: eleitor_id, eleicao_id, candidato_id (opcional) e em_branco (opcional,

default=False). Implemente validate() para verificar: (a) eleição existe e está com status='aberta'; (b) data atual está entre data_inicio e data_fim; (c) eleitor está apto na eleição; (d) eleitor ainda não votou; (e) candidato (se fornecido) pertence à eleição; (f) exatamente um entre candidato_id e em_branco=True foi informado.

Questão 03 — ViewSets, Filtros e Rotas

Commit esperado: viewsets, filtros e configuração do Swagger

Crie os ViewSets no arquivo urna/views.py e registre as rotas em eleicoes_api/urls.py:

17. EleitorViewSet — ModelViewSet com busca por nome, email, cpf; filtro por ativo.
18. EleicaoViewSet — ModelViewSet com filtros por status, tipo e criada_por; busca por titulo; ordenação por data_inicio.
19. CandidatoViewSet — ModelViewSet com filtro por eleicao; busca por nome, nome_urna, partido_ou_chapa. Otimize com select_related('eleicao').
20. AptidaoEleitorViewSet — ModelViewSet com filtros por eleitor e eleicao. Otimize com select_related('eleitor', 'eleicao').
21. RegistroVotacaoViewSet — ReadOnlyModelViewSet (apenas list e retrieve) com filtros por eleicao e ordenação por data_hora decrescente.
22. VotoViewSet — ReadOnlyModelViewSet com filtro por eleicao. NÃO deve permitir nenhuma operação de escrita, mesmo para admins.
23. Registre todas as rotas no DefaultRouter com os prefixos: eleitores, eleicoes, candidatos, aptidoes, registros-votacao, votos.
24. Configure o Swagger (drf-yasg) em /swagger/ e o ReDoc em /redoc/.

Questão 04 — Endpoint de Votação e Comprovante

Commit esperado: endpoint de votacao com comprovante e QR code

4.1 — Endpoint: Registrar Voto

Implemente via `@action` no `EleicaoViewSet`:

```
POST /eleicoes_api/eleicoes/{id}/votar/
```

Corpo da requisição (JSON):

```
{ "eleitor_id": 7, "candidato_id": 3} // ou, para voto em branco: { "eleitor_id": 7, "em_branco": true}
```

Comportamento obrigatório do endpoint:

- Validar a entrada com `VotacaoInputSerializer` (Questão 02).
- Criar o `RegistroVotacao` (eleitor + eleição). Se já existir (`IntegrityError` do `unique_together`), retornar HTTP 409 Conflict com a mensagem 'Eleitor já votou nesta eleição'.
- Gerar token com `secrets.token_urlsafe(32)`
- Criar o `Voto` com `eleicao`, `candidato` (ou `em_branco=True`) e `comprovante_hash`.
- Gerar a imagem do QR Code (use `qrcode` + `Pillow`) codificando uma URL como `/verificar-comprovante/?token=<TOKEN>`.
- Retornar HTTP 201 com a resposta abaixo. O token original aparece nesta resposta UMA ÚNICA VEZ.

Exemplo de resposta esperada:

```
HTTP 201 Created { "mensagem": "Voto registrado com sucesso. Guarde o seu comprovante.", "comprovante": { "token": "8N4q0aPxRz_KbS3vF7Tg9hYmJpL2WdRcXeUiHkZoVj0", "eleicao": "Eleição para o Diretório Acadêmico 2026", "candidato": "ANA (#13)", "data_hora": "2026-05-13T14:32:08Z", "qr_code_url": "/eleicoes_api/comprovalides/qr/?token=8N4q0aPxRz_..." } }
```

4.2 — Endpoint: Verificação de Comprovante

Implemente uma view (pode ser `@api_view`) acessível publicamente:

```
GET /eleicoes_api/verificar-comprovante/?token=<TOKEN>
```

Comportamento:

- Buscar `Voto` com `comprovante_hash` = hash calculado.
- Se encontrado: retornar { `eleicao`, `candidato` (ou 'BRANCO'), `data_hora`, `valido: true` }.
- Se não encontrado: HTTP 404 com { `valido: false`, `mensagem: 'Comprovante inválido'` }.

4.3 — Endpoint: Geração de QR Code

Implemente view auxiliar para o QR Code:

```
GET /eleicoes_api/comprovalides/qr/?token=<TOKEN>
```

- Gerar dinamicamente o PNG do QR Code apontando para a URL de verificação.
- Retornar `HttpResponse` com `content_type='image/png'`.

Questão 05 — Endpoints de Operação e Relatórios

Commit esperado: relatorios e gestao do ciclo da eleicao

5.1 — Endpoint: Abrir Eleição

Implemente via `@action` no `EleicaoViewSet`:

POST `/eleicoes_api/eleicoes/{id}/abrir/`

- Validar que status atual é 'rascunho'.
- Validar que a eleição tem pelo menos 2 candidatos cadastrados.
- Validar que tem pelo menos 1 eleitor apto.
- Alterar status para 'aberta' e retornar a eleição atualizada.

5.2 — Endpoint: Encerrar Eleição

Implemente via `@action` no `EleicaoViewSet`:

POST `/eleicoes_api/eleicoes/{id}/encerrar/`

- Validar que status atual é 'aberta'.
- Alterar status para 'encerrada'. A partir daqui o endpoint `/votar/` deve recusar votos.

5.3 — Endpoint: Apuração (Resultado da Eleição)

Implemente via `@action` no `EleicaoViewSet`:

GET `/eleicoes_api/eleicoes/{id}/apuracao/`

- Permitido apenas se status='encerrada' ou 'apurada' — caso contrário, HTTP 403.
- Calcular: total de votos válidos (em candidatos), total de votos em branco e total de abstenções (`aptos - registros_votacao.count()`).
- Para cada candidato: total de votos e percentual calculado sobre o total de votos VÁLIDOS.
- Ordenar candidatos por total de votos decrescente e identificar o(s) vencedor(es) (lidar com empate).
- Se status era 'encerrada', mudar para 'apurada' ao final.

Exemplo de resposta esperada (HTTP 200):

```
{ "eleicao": "Eleição para o Diretório Acadêmico 2026", "total_aptos": 320,
"total_votantes": 287, "total_abstencoes": 33, "votos_validos": 271, "votos_branco":
16, "comparecimento_pct": 89.69, "resultado": [ { "posicao": 1, "candidato": "ANA",
"numero": 13, "votos": 142, "percentual": 52.40 }, { "posicao": 2, "candidato":
"BRUNO", "numero": 22, "votos": 95, "percentual": 35.06 }, { "posicao": 3,
"candidato": "CARLA", "numero": 31, "votos": 34, "percentual": 12.55 } ], "vencedores":
["ANA"], "houve_empate": false}
```

5.4 — Endpoint: Lista de Votantes (Auditoria de Comparecimento)

Implemente via `@action` no `EleicaoViewSet`:

GET `/eleicoes_api/eleicoes/{id}/votantes/`

- Listar quem COMPARECEU à eleição: lista de eleitores com nome, cpf (parcialmente mascarado) e data_hora do registro.

- NÃO retornar qualquer informação sobre em quem cada um votou.
- Permitir filtro ?compareceu=false para listar quem se ABSTEVE (eleitores aptos sem registro de votação).

5.5 — Endpoint: Cadastro em Lote de Aptidões

Implemente via @action no EleicaoViewSet:

```
POST /eleicoes_api/eleicoes/{id}/cadastrar-aptos/
```

Corpo da requisição:

```
{ "eleitores_ids": [1, 2, 3, 7, 12, 45] }
```

- Permitido apenas se eleição está em 'rascunho'.
- Criar uma AptidaoEleitor para cada eleitor da lista.
- Retornar { total_cadastrados: N }.

5. Roteiro de Testes

Após implementar todas as questões, teste a API usando uma das ferramentas vistas em aula: Swagger UI, REST Client (VS Code) ou Insomnia/Postman.

5.1 Casos de Teste Mínimos

#	Ação	Método	Endpoint	Resultado Esperado
1	Criar Eleitor	POST	/eleitores/	HTTP 201
2	Criar Eleição (rascunho)	POST	/eleicoes/	HTTP 201
3	Criar Candidatos (2+)	POST	/candidatos/	HTTP 201
4	Cadastrar aptos em lote	POST	/eleicoes/1/cadastrar-aptos/ /	HTTP 200
5	Abrir eleição sem candidatos	POST	/eleicoes/2/abrir/	HTTP 400
6	Abrir eleição válida	POST	/eleicoes/1/abrir/	HTTP 200 — status='aberta'
7	Registrar voto válido	POST	/eleicoes/1/votar/	HTTP 201 — retorna token e qr_code_url
8	Votar duas vezes (mesmo eleitor)	POST	/eleicoes/1/votar/	HTTP 409 Conflict
9	Votar como inapto	POST	/eleicoes/1/votar/	HTTP 400
10	Voto em branco	POST	/eleicoes/1/votar/	HTTP 201
11	Verificar comprovante válido	GET	/verificar-comprovante/?token=...	HTTP 200 — valido=true
12	Verificar comprovante inválido	GET	/verificar-comprovante/?token=xxx	HTTP 404
13	Baixar QR Code (PNG)	GET	/comprovantes/qr/?token=...	HTTP 200 — image/png
14	Encerrar eleição	POST	/eleicoes/1/encerrar/	HTTP 200
15	Votar após encerramento	POST	/eleicoes/1/votar/	HTTP 400
16	Apurar eleição	GET	/eleicoes/1/apuracao/	HTTP 200 — JSON com percentuais
17	Listar votantes	GET	/eleicoes/1/votantes/	HTTP 200 — sem revelar votos

#	Ação	Método	Endpoint	Resultado Esperado
18	Tentar POST direto em /votos/	POST	/votos/	HTTP 405 Method Not Allowed

6. Entrega e Critérios

6.1 O que deve ser entregue

- Repositório do GitHub Classroom (link na capa) com TODOS os commits descritos na Seção 7.
- Arquivo requirements.txt gerado com pip freeze (incluindo Django, djangorestframework, drf-yasg, qrcode, Pillow, psycopg2-binary).
- Arquivo api_eleicoes.http com os 18 casos de teste do REST Client.
- Arquivo README.md com: (a) descrição da estratégia de anonimato adotada, (b) instruções de instalação, (c) comandos de migração, (d) instruções de execução, (e) link do Swagger.
- Diagrama do modelo de dados (django-extensions graph_models, dbdiagram.io ou drawio) anexado ao README.md.

6.2 Critérios de Avaliação

Critério	Peso
Q01 — Setup do projeto, modelos com clean() e migrações	15%
Q02 — Serializers com campos calculados, validações e VotacaoInputSerializer	15%
Q03 — ViewSets, filtros, otimizações e Swagger (incluindo VotoViewSet sem escrita)	15%
Q04 — Endpoint de votação, comprovante hash e QR code	25%
Q05 — Endpoints de gestão do ciclo (abrir/encerrar/apurar/votantes/aptos em lote)	20%
README.md, requirements.txt e arquivo .http para teste dos endpoints	10%

7. Commits e Cronograma de 3 Horas

A avaliação considera o histórico de commits. Cada questão deve gerar PELO MENOS um commit com a mensagem padronizada abaixo. Commits adicionais (refactor, fix, docs) são bem-vindos, mas os 5 commits principais devem existir e estar na ordem correta.

7.1 Sequência obrigatória de commits

#	Etapa	Mensagem de commit obrigatória
1	Setup + Modelagem	setup do projeto e modelos iniciais
2	Serializers	serializers da API
3	ViewSets + Rotas + Swagger	viewsets, filtros e configuração do Swagger
4	Votação + Comprovante + QR	endpoint de votacao com comprovante e QR code
5	Relatórios + Ciclo da Eleição	relatorios e gestao do ciclo da eleicao
6	Testes + README	docs: README, diagrama e arquivo .http de testes

OBSERVAÇÃO — Faça push após CADA commit. O servidor do GitHub Classroom registra os timestamps e eles serão usados para confirmar que o trabalho foi feito dentro da janela de 3 horas.

7.2 Sugestão de Distribuição do Tempo (3h)

Bloco	Atividade	Duração sugerida
00:00 — 00:35	Q01: Setup, modelos, migrações e Admin	35 min
00:35 — 01:05	Q02: Serializers (incluindo VotacaoInputSerializer)	30 min
01:05 — 01:30	Q03: ViewSets, rotas e Swagger	25 min
01:30 — 02:20	Q04: Endpoint de votação + comprovante + QR (parte mais densa)	50 min
02:20 — 02:45	Q05: Endpoints de ciclo e apuração	25 min
02:45 — 03:00	Testes (.http), README e ajustes finais	15 min